

가천대학교 스마트시티학과 – 가상공간과 메타버스

Lecture 13 – Public Services, Infrastructure and Environment Lab

Chris Ohk

utilForever@gmail.com

학습 목표

- Lecture 12의 공공서비스 / 자원의 변화를 기존 도시 프로그램에 적용한다.
- 함수 6개를 작성해 권역 / 대기 / 자원 흐름을 계산한다.
  - 작성 → 단계 검사 → 화면 확인 → 조건을 바꿔 실험

실행 준비

- 압축을 푼 스켈레톤 폴더에서 프로그램을 실행한다.
  - Windows: `py -m tkinter` → `py app.py`
  - macOS / Linux: `python3 -m tkinter` → `python3 app.py`
- `service_rules.py`를 저장한 뒤 앱을 닫고 다시 실행한다.

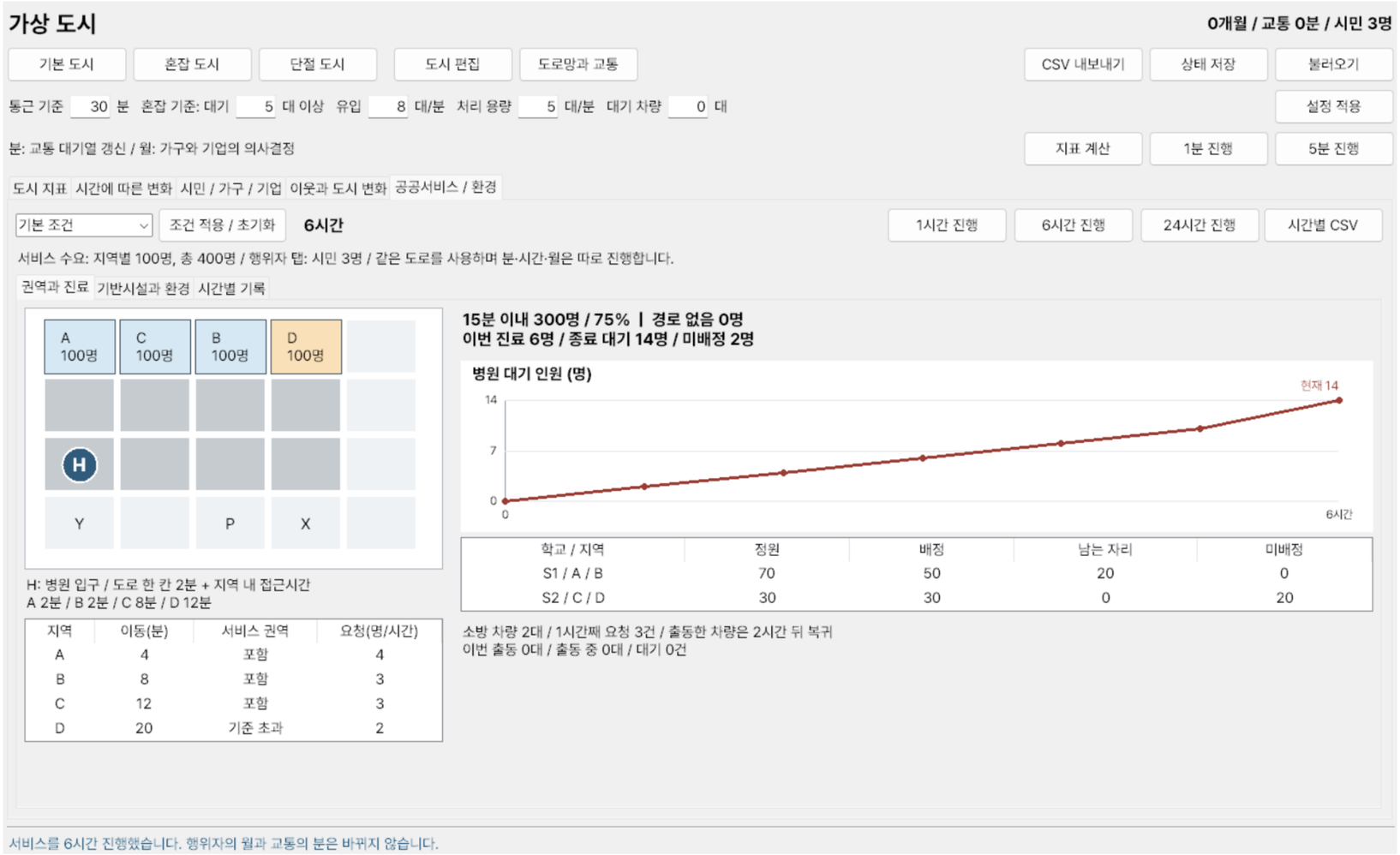
역할 나누기

- 제공된 코드에서 TODO 주석 부분을 채운다.

파일	역할	수업 중 수정
service_rules.py	규칙 함수들 (권역 / 대기 / 전력 / 물 / 폐기물 / 오염)	직접 작성
service_gui.py	GUI 관련 (탭 / 지도 / 그래프 / 버튼)	제공
service_support.py	동작 관련 (시간 갱신 / 시나리오 / 기록 / 저장)	제공
app.py / 기존 도시 코드	프로그램 실행 / 기존 도시와 행위자	제공
checks_services.py	단계별 자동 검사	제공

실습 완성 화면

- 오늘 실습 코드를 완성하면 다음과 같은 결과를 얻을 수 있다.



공공서비스의 자료구조

- services에 현재 상태를 두고, 갱신 결과를 history에 기록한다.

자료	주요 항목	용도
city / transport / agents	지도 / 도로 / 시민과 가구	지난주 상태
services	hour / queue / water_stored	현재 시간 / 대기 / 물 저장량
services	battery_kwh / waste_city / pollution	배터리 / 폐기물 / 오염량
services["history"]	시간별 결과를 담은 딕셔너리 목록	시간별 비교 / CSV 저장

이번 실습의 도시

- 지역 A / B / C / D의 서비스 대상 인구를 각각 100명으로 정한다.
  - H : 병원 입구 / X와 Y : 기업 입지 / P : 공원

y / x	0	1	2	3	4
0	A 100명	C 100명	B 100명	D 100명	물
1	도로	도로	도로	도로	물
2	H 도로	도로	도로	도로	땅
3	Y	땅	P	X	땅

실습 1 : 서비스 권역

- `service_coverage(...)` 함수를 작성한다.
  - 지역별 이동시간과 인구를 입력받는다.
  - 기준 시간 15분 이내인 지역의 인구를 더한다.
  - 도달할 수 없는 지역의 인구는 따로 센다.
  - 권역 목록 / 권역 인구 / 인구 비율 / 도달 불가 인구를 반환한다.

지역별 서비스 권역

- 15분 이내에 도착할 수 있는 A / B / C의 인구를 더한다.

지역	이동시간	인구	권역 포함
A	4분	100명	포함
B	8분	100명	포함
C	12분	100명	포함
D	20분	100명	제외

직접 해보기 : 서비스 권역

- `service_rules.py`의 TODO 1을 완성해 보자.
  - `python checks_services.py --stage 1`
  - 권역 인구 300명 / 인구 비율 0.75
- 0분과 None을 같은 조건으로 처리하면 어떤 문제가 생길까?

서비스 권역 해설 : 인구 집계

- 지역별 이동시간을 확인하고 권역 인구와 도달 불가 인구를 센다.

```python
def service_coverage(travel_minutes, populations,
                     limit_minutes):
    covered = []
    covered_population = 0
    unreachable_population = 0

    for area, population in populations.items():
        minutes = travel_minutes[area]

        if minutes is None:
            unreachable_population += population
        elif minutes <= limit_minutes:
            covered.append(area)
            covered_population += population
```

서비스 권역 해설 : 비율과 반환값

- 권역 인구를 전체 인구로 나누고, 빈 도시의 비율은 0으로 정한다.

```
    total_population = sum(populations.values())
    ratio = (covered_population / total_population
             if total_population else 0)

    return {
        "covered": covered,
        "population": covered_population,
        "ratio": ratio,
        "unreachable_population": unreachable_population
    }
```

실습 2 : 진료 대기

- `advance_queue(...)` 함수를 작성한다.
  - 현재 대기 인원에 새로 도착한 인원을 더한다.
  - 처리 가능 인원을 넘지 않는 범위에서 진료한다.
  - 실제 진료 인원과 다음 시간의 대기 인원을 반환한다.

요청 인원과 실제 진료 인원

- 10명이 진료를 요청해도 한 시간에 8명까지만 진료한다.

계산 단계	계산	결과
진료 대상	대기 0명 $+$ 도착 10명	10명
실제 진료	$\min(10$명, 처리 가능 8명$)$	8명
다음 시간 대기	대상 10명 $-$ 진료 8명	2명

직접 해보기 : 진료 대기

- `service_rules.py`의 TODO 2를 완성해 보자.
  - `python checks_services.py --stage 2`
  - 실제 진료 8명 / 다음 시간 대기 2명
- 처리 가능 인원보다 진료 대상이 적으면 몇 명을 진료할까?

진료 대기 해설

- 진료 대상과 처리 가능 인원 중 작은 값을 실제 진료 인원으로 정한다.

```python
def advance_queue(queue, arrivals, capacity):
    served = min(queue + arrivals, capacity)
    return {"served": served,
            "queue": queue + arrivals - served}
```

실습 3 : 전력 배분

- `allocate_power(...)` 함수를 작성한다.
  - 공급 전력을 우선순위 목록의 순서대로 배분한다.
  - 각 시설의 수요와 남은 공급량 중 작은 값을 배분한다.
  - 시설별 공급 전력을 딕셔너리로 반환한다.

우선순위에 따른 전력 배분

- 100kW를 병원 / 펌프 / 주거 / 산업 순서로 배분한다.

시설	수요 / kW	배분 / kW	남은 전력 / kW
병원	30	30	70
펌프	20	20	50
주거	50	50	0
산업	20	0	0

공급 전력이 달라지면

- 전력 부족은 총량과 시설별 배분을 함께 확인한다.

공급 / kW	병원	펌프	주거	산업
0	0	0	0	0
100	30	20	50	0
150	30	20	50	**20**

직접 해보기 : 전력 배분

- `service_rules.py`의 TODO 3을 완성해 보자.
  - `python checks_services.py --stage 3`
  - 병원 30 kW / 펌프 20 kW / 주거 50kW / 산업 0kW
- 우선순위를 바꾸면 어느 시설의 공급량이 달라질까?

전력 배분 해설

- 한 시설에 배분할 때마다 남은 전력에서 그 양을 뺀다.

```python
def allocate_power(supply_kw, demands_kw, priority):
    allocated = {}
    remaining = supply_kw

    for facility in priority:
        allocated[facility] = min(
            demands_kw[facility], remaining)
        remaining -= allocated[facility]

    return allocated
```

실습 4 : 물탱크

- `update_water(...)` 함수를 작성한다.
  - 시작 저장량에 이번 시간의 유입량을 더한다.
  - 사용 가능한 물을 수요만큼 공급한 뒤 남은 물을 저장한다.
  - 공급량 / 저장량 / 부족량 / 넘친 양을 반환한다.

물의 공급과 저장

- 시작 저장량 10m³에 6m³가 들어오고, 8m³를 공급한다.

계산 단계	계산	결과 / m³
사용 가능한 물	저장 10 + 유입 6	16
실제 공급량	$\min(\text{수요 } 8, \text{사용 가능 } 16)$	8
종료 저장량	$\min(\text{남은 } 8, \text{용량 } 20)$	8
부족량 / 넘친 양	미공급 수요 / 저장 용량 초과	0 / 0

직접 해보기 : 물탱크

- `service_rules.py`의 TODO 4를 완성해 보자.
  - `python checks_services.py --stage 4`
  - 종료 저장량 8m³ / 실제 공급량 8m³
- 저장 용량을 초과한 물은 어떤 값에 기록해야 할까?

물탱크 해설

- 공급한 뒤 남은 물을 저장하고 부족한 양과 넘친 양을 따로 기록한다.

```python
def update_water(stored_m3, inflow_m3,
                 demand_m3, capacity_m3):
    available = stored_m3 + inflow_m3
    supplied = min(demand_m3, available)
    remaining = available - supplied
    overflow = max(0, remaining - capacity_m3)

    return {"stored": min(remaining, capacity_m3),
            "supplied": supplied,
            "unmet": demand_m3 - supplied,
            "overflow": overflow}
```

실습 5 : 폐기물

- `update_waste(...)` 함수를 작성한다.
  - 도시에 남은 폐기물에 새로 발생한 양을 더한다.
  - 수거한 폐기물을 처리장으로 옮긴 뒤 처리한다.
  - 도시와 처리장의 잔량 / 수거량 / 처리량을 반환한다.

폐기물의 수거와 처리

- 발생량 8t 중 6t를 수거하고, 수거한 양 중 5t를 처리한다.

장소	들어온 양 / t	나간 양 / t	남은 양 / t
시내	발생 8	수거 6	**2**
처리장	수거 6	처리 5	**1**
전체	발생 8	처리 5	3

직접 해보기 : 폐기물

- `service_rules.py`의 TODO 5를 완성해 보자.
  - `python checks_services.py --stage 5`
  - 도시 잔량 2t / 처리장 잔량 1t
- 수거량만 늘리면 전체 폐기물 잔량도 줄어들까?

폐기물 해설

- 수거량은 시내에서 빼고 처리장에 더하며, 처리량은 처리장에서 뺀다.

```python
def update_waste(city_t, plant_t, generated_t,
                 collection_t, treatment_t):
    collected = min(city_t + generated_t, collection_t)
    processed = min(plant_t + collected, treatment_t)

    return {"city": city_t + generated_t - collected,
            "plant": plant_t + collected - processed,
            "collected": collected,
            "processed": processed}
```

실습 6 : 오염물질

- `transport_pollution(...)` 함수를 작성한다.
  - 모든 셀의 이동량은 갱신 전 상태로 계산한다.
  - 동쪽으로 20%를 이동시킨 뒤 각 셀에서 10%를 제거한다.
  - 마지막 셀은 경계 밖으로 내보내지 않고 새 목록을 반환한다.

셀 사이의 이동과 제거

- 동쪽으로 20%를 옮긴 뒤 각 셀에 남은 양의 10%를 제거한다.

단계	첫 번째 셀	두 번째 셀	세 번째 셀	합계
갱신 전	100	0	0	100
이동 후	80	20	0	100
제거 후	72	18	0	90

다음 상태와 입력 목록의 구분

- 초깃값이 [100, 0, 0]이면 첫 갱신 후 세 번째 셀의 값은 0이다.

입력 / 이동 / 제거	반환 목록	합계
`[100, 0, 0] / 20% / 10%`	`[72, 18, 0]`	90
`[100, 0, 0] / 20% /  0%`	`[80, 20, 0]`	100
`[0, 0, 100] / 20% / 10%`	`[0, 0, 90]`	90

직접 해보기 : 오염물질

- `service_rules.py`의 TODO 6을 완성해 보자.
  - `python checks_services.py --stage 6`
  - 다음 상태 `[72, 18, 0]` / 합계 90
- 입력 목록을 바로 바꾸면 다음 셀의 이동량은 어떻게 될까?

오염물질 해설

- 갱신 전 목록에서 이동량을 계산한 뒤 새 목록에 반영한다.

```python
def transport_pollution(amounts, move_ratio,
                        removal_ratio):
    moved = amounts.copy()

    for index in range(len(amounts) - 1):
        transfer = amounts[index] * move_ratio
        moved[index] -= transfer
        moved[index + 1] += transfer

    return [amount * (1 - removal_ratio)
            for amount in moved]
```

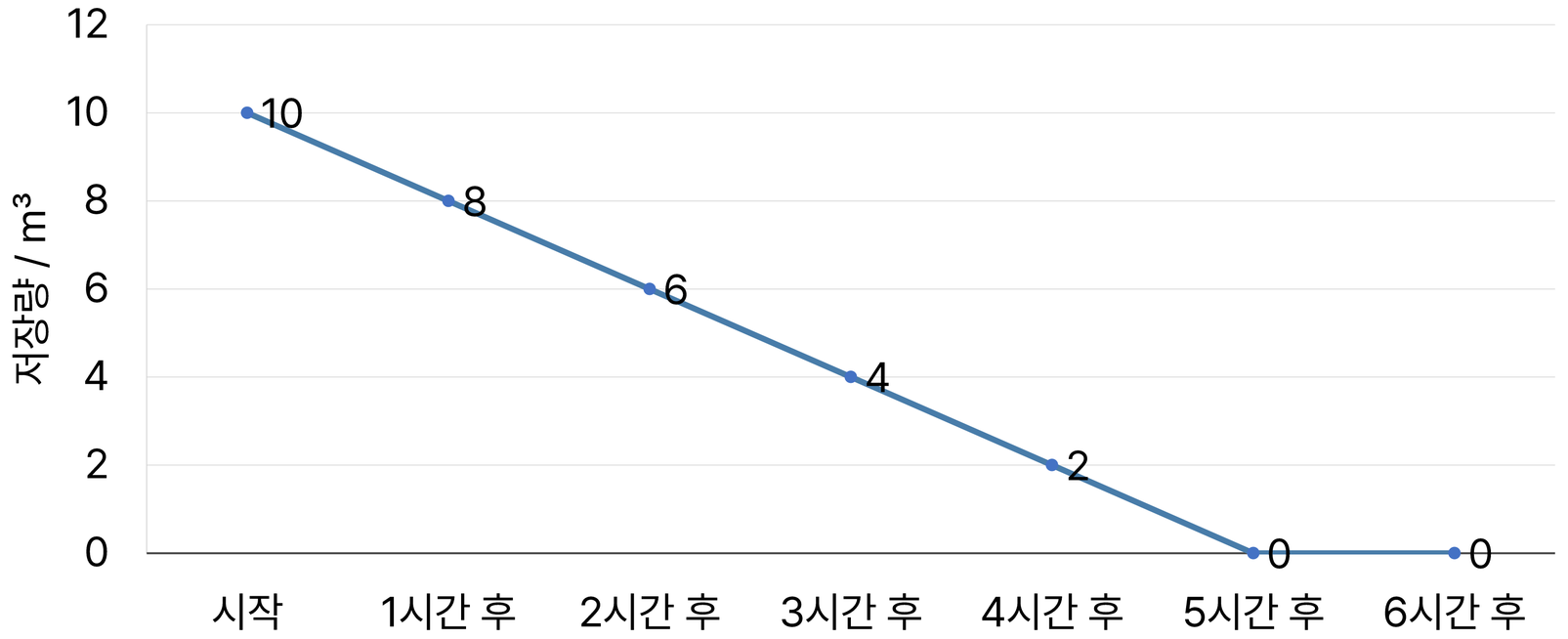
한 시간의 상태 갱신

- 경로와 자원 상태를 계산한 뒤 시설을 운영하고 결과를 기록한다.

순서	한 시간의 상태 갱신
1	재난 조건과 도로 연결 상태를 확인
2	전력 배분과 물 공급을 계산
3	실제 처리 가능 인원과 남은 대기 인원을 계산
4	오염 상태를 갱신하고 시간별 결과를 기록

비교 실험 1 : 기본 조건

- 기본 조건을 초기화하고 6시간 진행해 물탱크와 진료 대기를 확인한다.



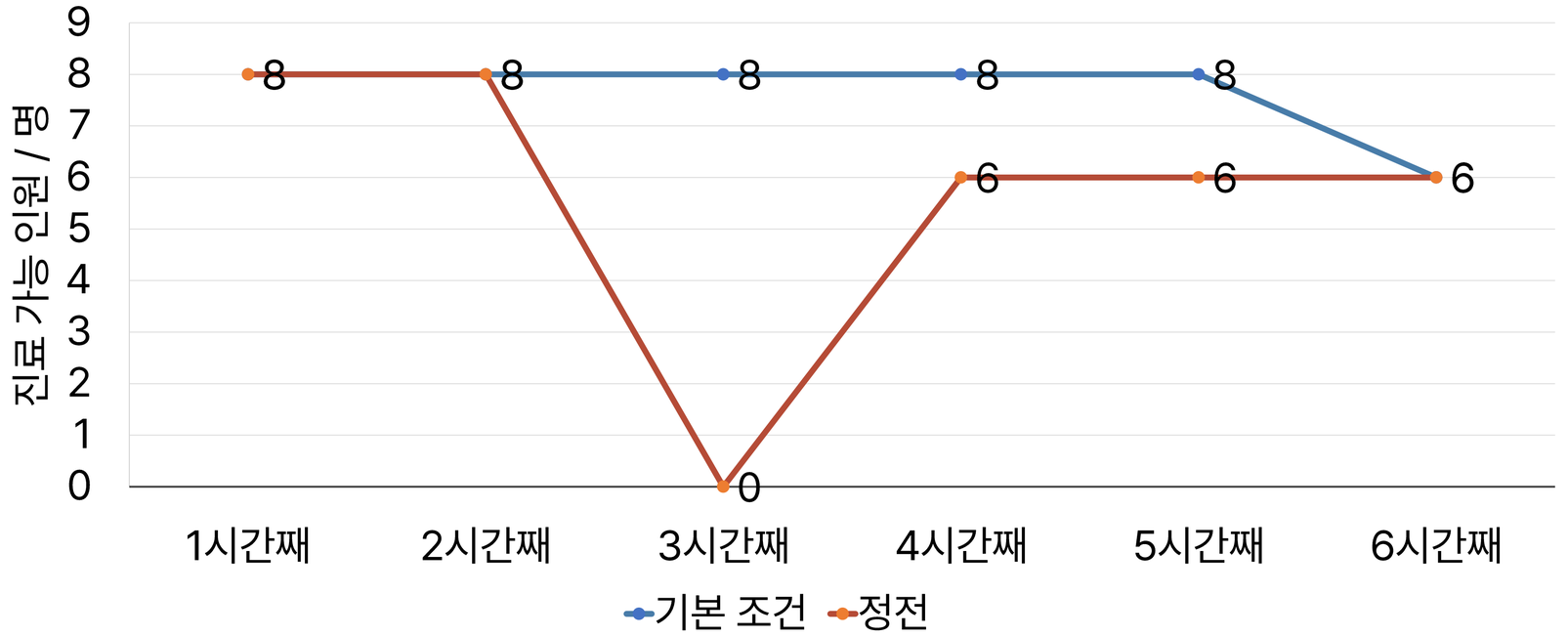
비교 실험 2 : 정전

- 정전 조건을 초기화하고, 2~3시간째 병원의 진료 가능 인원을 확인한다.

구간	계통 전력 / kW	배터리 잔량 / kWh	급수 / m^3	진료 가능 / 명
1시간째	100	60	8	8
2시간째	0	30	8	8
3시간째	0	0	0	0
4시간째	100	0	6	6

복구 뒤에도 남는 영향

- 정전이 끝난 4시간째에도 진료는 원래 처리 가능 인원의 75%에 그친다.



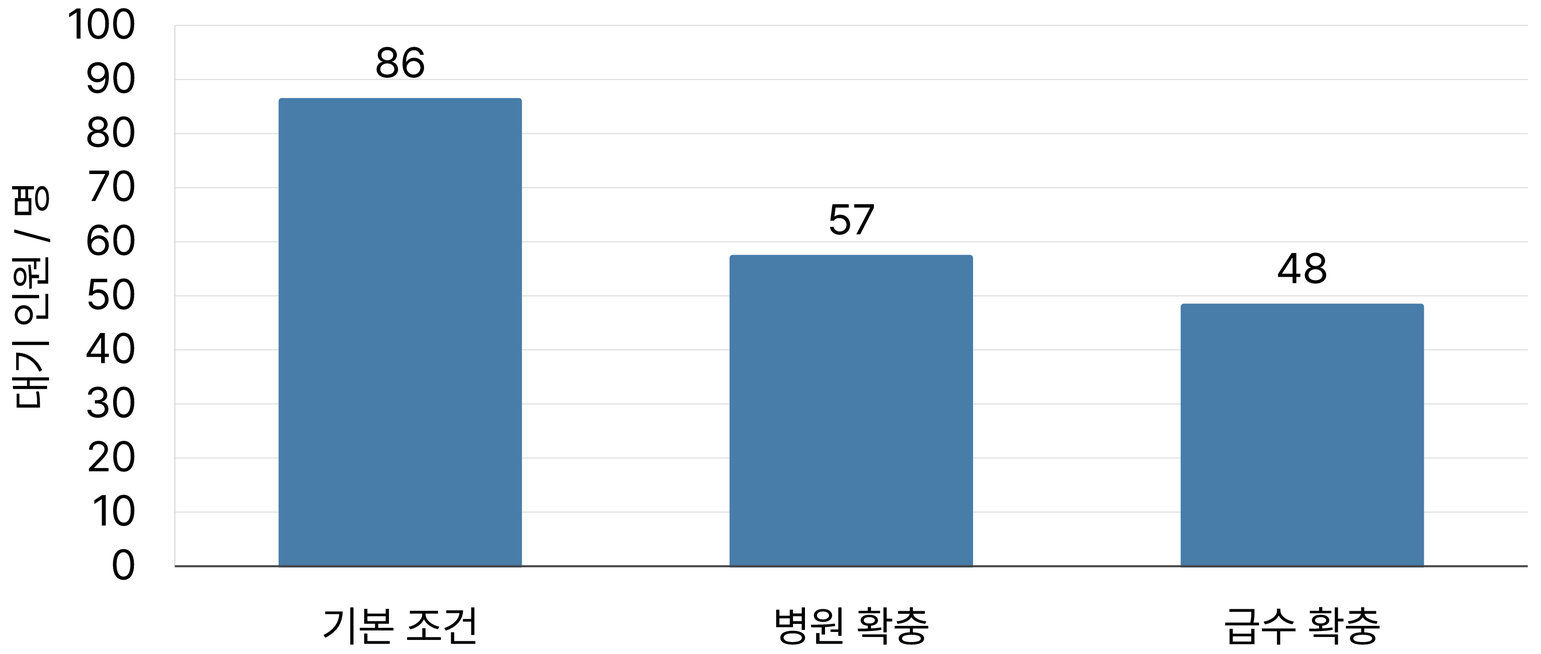
비교 실험 3 : 침수

- 권역 내 인구 비율은 같지만 D 지역으로 가는 경로가 끊긴다.

시점	D까지 시간	권역 비율	도달 불가 인구
1시간째	20분	75%	0명
2~3시간째	경로 없음	75%	100명
4시간째	20분	75%	0명

비교 실험 4 : 시설 확충

- 각 대안을 초기화한 뒤 24시간 후의 진료 대기 인원을 비교한다.



제공된 규칙 관찰

- 추가 코드 작성 없이 시설별 결과와 환경 조건을 확인한다.

대상	관찰할 결과
학교	S1은 20명분의 자리가 남고, S2는 20명이 미배정
소방	차량 2대 / 요청 3건 → 1건 대기 / 2시간 후 복귀
소음	서로 독립인 70dB 음원 두 개 → 약 73dB
녹지	같은 비에서 포장 40m³ / 녹지 20m³ 유출

CSV에 기록된 시계열

- 기본 조건을 6시간 진행하면, CSV에 1~6시간의 기록 6행이 저장된다.

시간 / h	진료 / 명	대기 / 명	물 저장 / m³	물 부족 / m³
1	8	2	8	0
2	8	4	6	0
3	8	6	4	0
4	8	8	2	0
5	8	10	0	0
6	6	14	0	2

직접 해보기 : 저장한 도시 복원

- 기본 조건을 6시간 진행한 뒤 상태 저장과 시간별 CSV를 실행한다.
- 한 번 더 진행한 7시간 결과를 적고 프로그램을 종료한다.
- 저장한 JSON을 불러와 6시간 상태가 돌아오는지 확인한다.
- 다시 1시간 진행해 앞서 적은 7시간 결과와 비교한다.

오늘 구현한 도시

- 이동시간으로 서비스 권역을 정하고 진료 대기 인원을 계산했다.
- 전력과 물을 공급하고, 폐기물과 오염량의 변화를 계산했다.
- 정전 / 침수 / 시설 확충의 결과를 같은 조건에서 비교했다.
- 시간별 결과를 기록하고 도시 상태를 저장해 실행을 이어 갔다.

감사합니다.

utilForever@gmail.com

https://github.com/utilForever

Facebook, Twitter, Instagram, LinkedIn: utilForever